

CPUlator ARM Assembly Program Trace and Analysis

Submission Task

Program

```
global _start

_start:
    mov r1, #18      @ r1 = ?
    mov r2, #6        @ r2 = ?
    add r3, r1, r2    @ r3 = ?
    sub r4, r1, r2    @ r4 = ?
    lsl r5, r2, #1    @ r5 = ?
    mul r6, r3, r2    @ r6 = ?
    add r7, r4, r5    @ r7 = ?
    b .              @ halt
```

Question 1 — Trace Table

The following table shows the register changed after each of the seven required instructions, along with its hexadecimal and decimal value.

Step	Instruction	Register Changed	Hex Value	Decimal Value
1	mov r1, #18	r1	0x00000012	18
2	mov r2, #6	r2	0x00000006	6
3	add r3, r1, r2	r3	0x00000018	24
4	sub r4, r1, r2	r4	0x0000000C	12
5	lsl r5, r2, #1	r5	0x0000000C	12
6	mul r6, r3, r2	r6	0x00000090	144
7	add r7, r4, r5	r7	0x00000018	24

Step-by-Step Calculations

Step 1

```
r1 = 18
18 = 0x12
```

Step 2

```
r2 = 6
6 = 0x06
```

Step 3

```
r3 = r1 + r2
    = 18 + 6
    = 24
    = 0x18
```

Step 4

```
r4 = r1 - r2
    = 18 - 6
    = 12
    = 0x0C
```

Step 5

```
r5 = r2 << 1
    = 6 << 1
    = 12
    = 0x0C
```

A left shift by 1 is equivalent to multiplying by 2: $6 \times 2 = 12$.

Step 6

```
r6 = r3 * r2
    = 24 * 6
    = 144
    = 0x90
```

Step 7

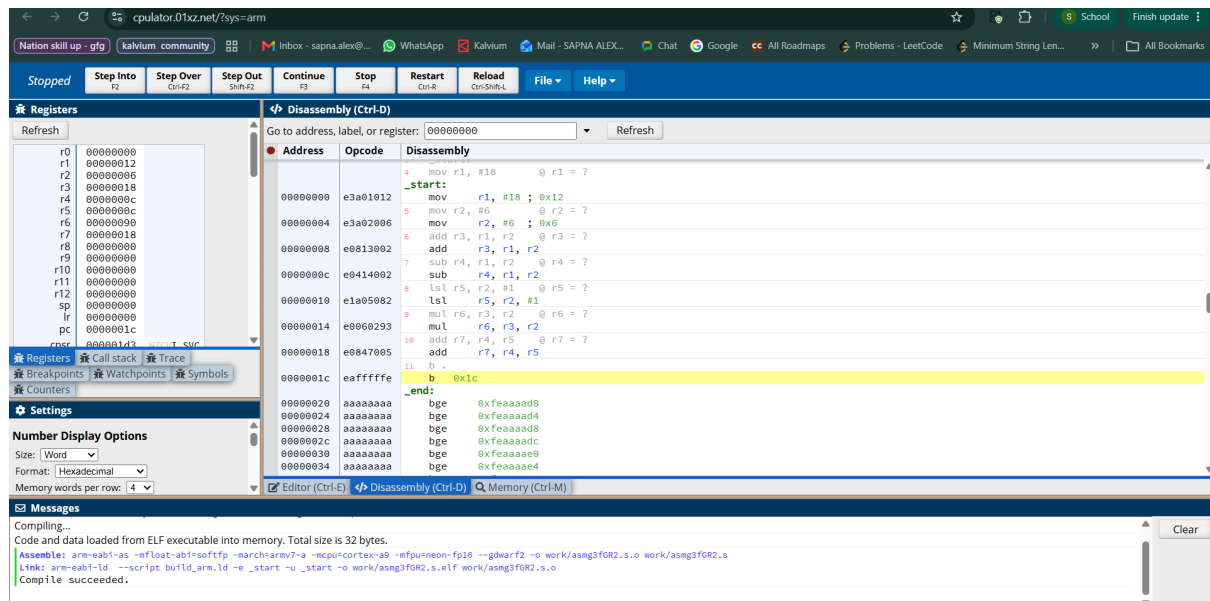
```
r7 = r4 + r5
    = 12 + 12
    = 24
    = 0x18
```

Final Register State

Register	Hex Value	Decimal Value
r1	0x00000012	18
r2	0x00000006	6
r3	0x00000018	24
r4	0x0000000C	12
r5	0x0000000C	12
r6	0x00000090	144
r7	0x00000018	24

CPUlator Final Register Screenshot

Insert the CPUlator Registers panel screenshot here, showing the final register state after all seven required instructions have executed.



Question 2 — Explain Step 6

The instruction in Step 6 is:

mul r6, r3, r2

At this point, r3 contains 24. The value of r3 was created in Step 3 by the instruction add r3, r1, r2.

Before Step 3:

r1 = 18

r2 = 6

Therefore:

$$\begin{aligned}
 r3 &= r1 + r2 \\
 &= 18 + 6 \\
 &= 24
 \end{aligned}$$

When Step 6 executes, the processor multiplies r3 by r2:

$$\begin{aligned}
 r6 &= r3 \times r2 \\
 &= 24 \times 6 \\
 &= 144
 \end{aligned}$$

Therefore, r3 holds 24 at Step 6 because the instruction add r3, r1, r2 in Step 3 calculated it by adding 18 and 6. This value is then used as a source operand by the multiplication instruction.

Question 3 — Instruction Order

If the order of instructions 6 and 7 is swapped, the final value of r7 would not change.

Original order:

```
mul r6, r3, r2
add r7, r4, r5
```

Swapped order:

```
add r7, r4, r5
mul r6, r3, r2
```

Before these instructions execute, the relevant register values are:

```
r2 = 6
r3 = 24
r4 = 12
r5 = 12
```

The add instruction calculates:

```
r7 = r4 + r5
    = 12 + 12
    = 24
```

The mul instruction calculates:

```
r6 = r3 × r2
    = 24 × 6
    = 144
```

The multiplication only changes r6. It does not change r4, r5, or r7. Therefore, r7 remains 24 whether the multiplication is executed before or after the addition.

Conclusion

The final value of r7 would not change because instructions 6 and 7 are independent of each other. Instruction 6 modifies r6, while instruction 7 uses r4 and r5 to calculate r7. Neither instruction modifies a register required by the other.

Final Answers Summary

Question	Answer	Key Value
Question 1	Final register state after 7 instructions	r1=18, r2=6, r3=24, r4=12, r5=12, r6=144, r7=24

Question 2	r3 was created by ADD in Step 3 and is used by MUL in Step 6.	r3 = 24
Question 3	Swapping instructions 6 and 7 does not affect r7.	r7 = 24